

Backprop Through a Transformer

One loss number, 124 million gradients · video 2:35

This lesson answers one question: how does one backward pass give every weight of GPT-2 its gradient, and can we trust it? Backprop is the **chain rule**, applied from the loss back to the weights. We check autograd against **finite differences** on a real GPT-2 weight (they agree to 7 digits, once a precision trap is avoided), look at gradient sizes layer by layer, and measure the cost: the backward pass takes about **2×** the time of the forward pass, and the saved activations take **3×** the memory of the weights.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The chain rule	▶ 0:08-0:40	2	☐
2. Checking autograd on GPT-2	▶ 0:40-1:13	2	☐
3. Every weight, and gradient sizes	▶ 1:13-1:45	2	☐
4. The cost: time and memory	▶ 1:45-2:25	2	☐

Before you start: Foundations (derivatives, the chain rule) and Deep Dive episode 17 (the loss and its gradient). Code: `code/d18_backprop` (GPT-2 small, about 500 MB; uses about 4 GB of RAM).

CONCEPT 1 OF 4

The chain rule

▶ Video 0:08–0:40

Training needs, for each of GPT-2's 124,439,808 weights, the gradient: how the loss would change if that weight moved a little. **Backprop** computes them all in one backward pass by applying the **chain rule** step by step, from the loss back to the weights.

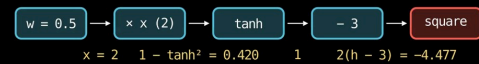
124,439,808 weights

for each: how would the loss change if it moved a little?

one backward pass computes all of them

▶ 0:21 124,439,808 weights, and for each, how the loss would change: one backward pass computes them all.

the chain rule, step by step from the loss back



by hand: -3.760292
autograd: -3.760291

▶ 0:39 $\text{loss} = (\tanh(w \cdot x) - 3)^2$: multiply the local derivatives, from the loss back. By hand -3.760292 , autograd -3.760291 .

A tiny example: $\text{loss} = (\tanh(w \cdot x) - 3)^2$, with $w = 0.5$ and $x = 2$. The local derivatives are $2(h - 3) = -4.477$ for the square, 1 for “ $- 3$ ”, $1 - \tanh^2 = 0.420$ for $\tanh$, and $x = 2$ for the multiplication. Their product, **-3.760292** , is what autograd computes (-3.760291 , float32 rounding).

KEY IDEA

The gradient of the loss with respect to a weight = the product of the local derivatives along the path from that weight to the loss (summed over paths).

Check 1: The chain rule

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

Same function, but $w = 0$ ($x = 2$). What is the gradient?

1.2 SHORT ANSWER

Why does backprop run from the loss backward rather than from each weight forward?

CONCEPT 2 OF 4

Checking autograd on GPT-2

▶ Video 0:40–1:13

Take one real weight in GPT-2's layer 5 MLP, entry [100, 200]. Autograd says its gradient is **−0.0000200195**. The slow way: nudge the weight up and down by $\epsilon = 0.0001$ and compute $(\text{loss}(w + \epsilon) - \text{loss}(w - \epsilon)) / 2\epsilon$. In float64 the result is **−0.0000200195**: the same to 7 digits.

GPT-2, layer 5 MLP, weight [100, 200]

```
autograd:          -0.0000200195
(loss(w + ε) - loss(w - ε)) / 2ε,  ε = 0.0001
finite differences: -0.0000200195
```

first try: finite differences 0.00238 vs autograd -0.00002 ✗

the library's built-in loss rounds to float32:
changes this small are lost

▶ 0:58 Layer 5 MLP, weight [100, 200]: autograd −0.0000200195; finite differences −0.0000200195.

▶ 1:12 The first check failed: the library's built-in loss rounds to float32. Our own float64 loss fixed it.

One trap: the first time, the check failed (0.00238 vs −0.00002). The library's built-in loss converts the logits to float32, whose rounding (about 5×10^{-7} on a loss of 4.4) swamps a change of 4×10^{-9} . Computing the cross-entropy ourselves, in float64, fixed it.

KEY IDEA

Finite differences are a slow but independent check of gradients; they need high precision because the changes are tiny.

Check 2: Checking autograd on GPT-2

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

To get every gradient of GPT-2 by finite differences (two forward passes per weight), how many forward passes would you need?

2.2 HANDS-ON CODE

Try it yourself. Check another weight: entry [0, 0] of layer 0's attention (`model.transformer.h[0].attn.c_attn.weight`).

```
W = model.transformer.h[0].attn.c_attn.weight
g = W.grad[0, 0].item()
with torch.no_grad():
    W[0, 0] += eps; up = lm_loss(model, ids).item()
    W[0, 0] -= 2 * eps; down = lm_loss(model, ids).item()
    W[0, 0] += eps
print(g, (up - down) / (2 * eps))
```

CONCEPT 3 OF 4

Every weight, and gradient sizes

▶ Video 1:13-1:45

One backward pass gives every one of the 124,439,808 parameters its gradient, from a single loss number. Even all **50,257** rows of the token embedding get one, though at most 256 different tokens appear in the 256-token text: the same matrix also produces the output scores for every vocabulary entry.

one loss number → 124,439,808 gradients

token embedding: 50,257 of 50,257 rows get a gradient
(the same matrix also produces the output scores)

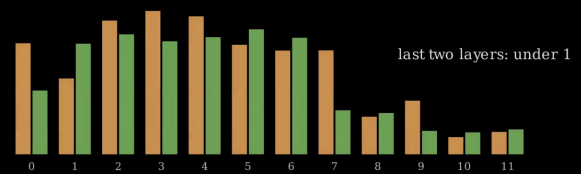
▶ 1:27 One loss number gives all 124,439,808 parameters a gradient, including all 50,257 embedding rows.

size of each layer's gradient (one backward pass)

nothing vanishes: layer 0 gets a large gradient

early and middle layers: ~2 to 4

attention MLP



▶ 1:43 Gradient size per layer: ~2 to 4 in the early and middle layers, under 1 in the last two.

Gradient sizes, layer by layer: the early and middle layers get the largest (attention 3.27 in layer 0, up to 4.23 in layer 3), the last two layers the smallest (0.51-0.73). Nothing vanishes on the way back: layer 0 still gets a large gradient, consistent with the residual stream's direct path (episode 13).

KEY IDEA

Backprop gives every parameter a gradient in one pass; in a pre-norm transformer the gradients reach the first layers intact.

Check 3: Every weight, and gradient sizes

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 TRUE OR FALSE

“Only the embedding rows of tokens that appear in the text get a gradient.”

True ☐ False ☐

3.2 MULTIPLE CHOICE

Which layers got the smallest gradients in this run?

- A. Layers 0-1
- B. Layers 3-4
- C. Layers 10-11
- D. All about equal

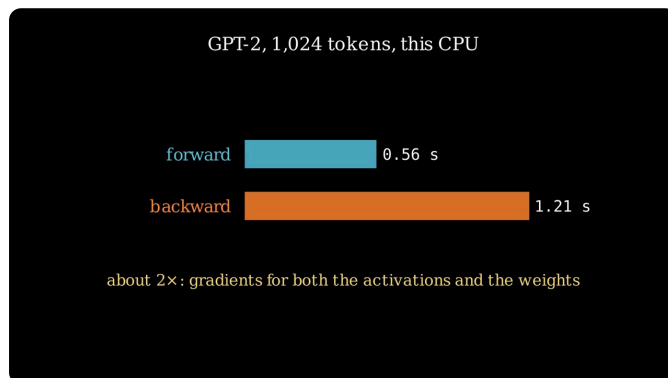
CONCEPT 4 OF 4

The cost: time and memory

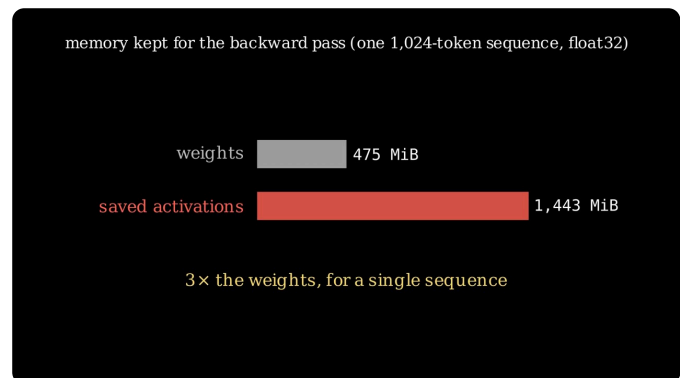
▶ Video 1:45–2:25

On this CPU, a forward pass over 1,024 tokens takes **0.56 s**; the backward pass **1.21 s**, about twice as long, because it computes gradients for both the activations (to keep going backward) and the weights (to update them).

To run backward, the forward pass must keep its intermediate results. Counting exactly what autograd saves (excluding the weights themselves): **1,443 MiB** of activations for one 1,024-token sequence, three times the 475 MiB of float32 weights. In code it is one call, `loss.backward()`; every parameter then holds its `.grad`, ready for the optimizer.



▶ 2:00 1,024 tokens on this CPU: forward 0.56 s, backward 1.21 s, about twice as long.



▶ 2:16 Activations saved for the backward pass: 1,443 MiB for one sequence, 3x the 475 MiB of weights.

KEY IDEA

Training costs about 3x a forward pass in compute (1 forward + ~2 backward) and needs memory for activations that grows with batch size and sequence length.

Check 4: The cost: time and memory

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

About how much activation memory would a batch of 8 such sequences need?

.....

4.2 CALCULATION

GPT-2 small has 124,439,808 float32 parameters. How many MiB is that?

.....

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The chain rule · rewatch ▶ 0:08-0:40

1.1 -12. $\tanh(0) = 0$: $2(0 - 3) \times 1 \times (1 - 0) \times 2 = -12$ (checked with autograd).

1.2 One loss and many weights: going backward reuses each intermediate derivative for every weight below it, so one pass gives all the gradients. Going forward would need one pass per weight.

2. Checking autograd on GPT-2 · rewatch ▶ 0:40-1:13

2.1 248,879,616. $2 \times 124,439,808$, against one forward and one backward pass with backprop.

2.2 0.0000307327 and 0.0000307328. Agreement to 6-7 digits again.

3. Every weight, and gradient sizes · rewatch ▶ 1:13-1:45

3.1 False (for GPT-2). GPT-2 ties the embedding to the output layer, and the softmax gives every vocabulary entry a gradient.

3.2 C. Attention 0.51 and 0.66, MLP 0.65 and 0.73.

4. The cost: time and memory · rewatch ▶ 1:45-2:25

4.1 About 11,544 MiB (11.3 GiB). $8 \times 1,443$ MiB: each sequence keeps its own activations.

4.2 About 475 MiB. $124,439,808 \times 4 \text{ bytes} / 2^{20} \approx 474.7$.